

Habit Gate

Product & Engineering Review

A clear path from personal prototype
to a dependable community release.

Prepared for	Quan and CEO review
Review baseline	macOS app 1.3.0, build 8; Chrome extension 1.0.0
Document revision	1.1 / 18 October 2026
Decision requested	Approve a focused hardening phase and private alpha; defer broad public release until the release gates pass.
Classification	Internal review. Screenshots include personal example habits.

The product promise

Complete your essential Habitify habits before opening selected social websites and distracting Mac apps. Keep the experience calm, understandable and lightweight.

This is an evidence-based review of the current implementation, not a claim of production readiness. Recommendations are explicitly marked as proposed. Revision 1.1 reflects the installed rule-timing update: target additions apply when saved; removals and habit edits wait until 4 AM.

Contents

1 Executive brief 3

2 Product scope and feature specifications 4

3 User journeys and application flow 8

4 Business rules and daily time model 10

5 State model and interaction semantics 11

6 Interface review and screenshot gallery 12

7 Architecture and responsibility boundaries 20

8 Data model, interfaces and invariants 21

9 Software engineering principles 22

10 Reliability and failure recovery 23

11 Security, privacy and trust boundaries 24

12 Verification and acceptance evidence 25

13 Performance and simplicity budget 26

14 Distribution, operations and release gates 27

15 Prioritized fixes and risk register 28

16 CEO review and decision worksheet 29

A Evidence index and provenance 30

B External references and terminology 31

C Engineering and support runbook 32

How to read this review

CEO: start with Sections 1–2, the screen review in Section 6, and the decisions in Section 16. **Engineering:** start with the feature specifications in Sections 2.1–2.3 and the sequence diagram in Section 3.1; then use Sections 4–5 and 7–15, the evidence index and runbook. The PDF has clickable contents, cross-references and source links.

1 Executive brief

RECOMMENDATION / PRIVATE ALPHA, WITH RELEASE GATES

Habit Gate is a local macOS companion to Habitify. It makes social access conditional on completing a small set of daily essentials. The core loop works on the owner's Mac: read Habitify, evaluate today's rules, block selected Chrome sites and hide selected Mac app windows until verified completion.

CEO recommendation

Keep the product small. Approve reliability, installation and trust improvements before adding features. The current evidence supports personal use and a controlled private alpha, after the critical recovery checks; it does not yet support broad public distribution.

What is already credible

- A native menu-bar app with a coherent interface, secure local credential storage and a real Habitify connection.
- Chrome website enforcement and a voluntary Mac app guard using the same daily completion model. Seven essentials and five apps are configured on the owner's Mac.
- Fifty-four passing automated tests: 46 Swift and eight browser tests. Immediate target additions and deferred removals have regression coverage. Real website redirects and a real Discord block have been observed. These are functional observations, not population-level reliability data.
- A restrained architecture with no external Swift or npm dependencies and no new Habit Gate cloud service. The stable version was retained after an optimization experiment showed no clear practical benefit. [E01–E04]

What prevents a dependable public release

1. **Installation is for a developer.** The app is locally ad-hoc signed; Chrome loads unpacked files in Developer mode. There is no proven public installer, update or recovery journey.
2. **Protection is voluntary and partial.** Other browsers, unconfigured profiles, extension removal or quitting the app can bypass it. Mac app windows are hidden; processes and notifications may continue.
3. **Several real-world transitions remain unverified.** Completion, undo, cross-device propagation, sleep/wake, redirects and already-open tabs still require a controlled live acceptance run.
4. **Failure recovery needs explicit review.** Corrupt settings and failed disk writes can make stored policy and in-memory state disagree. These are code-inspection risks, not demonstrated production incidents.
5. **The original RAM budget is not met.** The stable warm idle sample recorded 107.06 MB mean RSS against a target below 100 MB. CPU was 0.154% of one core, below the 0.5% target. [E02, E04, E10]

The next decision

Approve a Chrome-first private alpha focused on the gates in Section 14. Nominate a product owner and an engineering owner. Keep Safari, exercises, rewards and analytics deferred. Do not advertise tamper-resistant blocking or a guaranteed memory budget.

2 Product scope and feature specifications

User and job to be done

The first user is an adult Mac owner who already uses Habitify and wants a voluntary commitment device: “Help me complete my essentials before I open distractions.” This is not a parental-control, workplace-monitoring or device-security product. A commercial target segment, pricing and demand have not been validated.

ID	Current feature	Required behavior
F01	Habitify connection	Verify the catalog and dated journal before saving a key in Keychain. Perform GET requests only. Keep the key out of browser state.
F02	Essential habits	Select supported good habits with one active daily goal. Follow daily, weekday, month-date or interval repeat schedules. Recommend 2–4 essentials.
F03	Website rules	Default to Facebook, Instagram, X, Twitter and fb.com. Include subdomains. Support 1–50 validated domains; keep Habitify accessible. Additions apply when saved; removals wait until 4 AM.
F04	Daily gate	Every selected habit due on the active date must be completed. Verified rest days may unlock. Unknown or invalid habits stay blocked.
F05	Progress and status	Show access status, remaining habits, active date, last successful sync and browser coverage in the Today panel and main window.
F06	Helpful interruption	Blocked pages offer Open Habitify, Sync now and Go back. Offer Continue only after valid verified access.
F07	Mac app guard	Select up to 50 apps by bundle identifier. Hide their windows while blocked; preserve sessions. Additions apply when saved; removals wait until 4 AM. Protect Habitify and essential system apps from selection.
F08	Preferences	Optional login launch and unlock notification. Keep the app available from the menu bar after windows close.

[E01, E05–E08]

Quality requirements

Q01 Correctness: one dated policy drives native and browser decisions; expiry and undo must re-lock. **Q02 Trust:** explain coverage and errors without claiming whole-device protection. **Q03 Privacy:** local policy, minimal data sharing, no key in browser files or logs. **Q04 Responsiveness:** UI and enforcement continue while Keychain waits. **Q05 Resources:** idle CPU below 0.5% of one core and combined native RSS below 100 MB are targets, not current guarantees. **Q06 Usability:** a new user can understand status and finish setup without developer help; this requires a pilot study.

Explicit exclusions

No habit editing, weekly/monthly goal evaluation, exercise bypass, pause button, social feed, rewards, detailed analytics or Safari package. Mac app guarding is implemented; hard process denial is outside this product’s current scope.

2.1 Feature specification: connection and daily policy

NORMATIVE BEHAVIOR / VERIFICATION STATUS IN SECTION 12

The following specifications define observable behavior. Acceptance criteria are release checks, not claims that every scenario has been tested live. "Must" applies to the stated feature contract; additional proposed controls remain in the backlog.

F01 — Connect and verify Habitify

Actor/precondition: Mac owner with eligible Habitify API access. **Input:** a credential entered locally in the secure field. **Flow:** validate input, GET the catalog and active-date journal, save the verified key in Keychain, then expose a connected state. The native UI remains responsive during credential access.

Criterion	Given / when / expected result
AC-01a	Given a valid key, when both GET responses decode successfully and Keychain saves, connection is ready. Browser responses contain no credential.
AC-01b	Given an empty/malformed key or API rejection, verification fails with a sanitized explanation. A rejected candidate must not replace the existing saved key.
AC-01c	Given delayed/denied Keychain access, UI, daily expiry and cached app rules continue; Connection offers retry after failure.

F02 — Select supported essentials

Precondition: verified connection and loaded catalog. **Input:** nonempty set of habit IDs; no fixed maximum is currently imposed. Recommend 2–4 rather than requiring that count. Eligible selections are good habits with one active daily goal and a supported repeat schedule. Ended or missing selections become configuration issues during evaluation. **AC-02a:** unsupported weekly/monthly goals and bad habits cannot be activated through the selector. **AC-02b:** the active policy survives relaunch; later edits remain pending until 4 AM. **AC-02c:** a missing/archived/ended habit never silently becomes a rest-day unlock.

F04 — Evaluate, unlock, undo and reset

Trigger: successful cloud verification, day/clock event or cached-proof validation. **Output:** one dated, expiring access decision shared with adapters. Only `completed` satisfies a due essential; partial, skipped, failed or unknown states block. All selected habits must be valid for a rest-day unlock. **AC-04a:** all due essentials complete unlocks; one incomplete essential blocks. **AC-04b:** a fetched undo re-locks native and browser access. **AC-04c:** at 4 AM yesterday's proof cannot permit access. **AC-04d:** offline access survives only while a previously verified proof remains valid. **AC-04e:** habit edits and target removals cannot weaken today's policy before its boundary; target additions apply when saved without invalidating same-day habit proof. The native five-second check may introduce seconds of reaction time; quantify it in the live matrix.

2.2 Feature specification: websites and blocked page

F03 — Configure and protect website domains

Precondition: active or draft policy; Chrome add-on loaded in the intended profile. **Input:** 1–50 distinct normalized domains, including subdomains. Reject URL schemes, paths, ports, invalid labels and Habitify domains. Custom-domain coverage requires appropriate Chrome permissions. After activation, Save websites adds new domains to today's policy immediately; removals stay active until the next 4 AM. Chrome normally reads native changes within 30 seconds. Saving this screen must not commit an unsaved habit draft.

Criterion	Example / event	Expected result
AC-03a	`facebook.com`, `m.facebook.com`	Both match a Facebook rule.
AC-03b	`notfacebook.com`	Does not match Facebook; unrelated sites remain usable.
AC-03c	Mixed case, spaces, trailing dot or leading wildcard	Normalize consistently; deduplicate before the 50-domain bound.
AC-03d	Custom domain lacks permission	Report incomplete coverage and offer a narrow permission request. Do not imply the profile is fully protected.
AC-03e	Direct link, redirect or already-open selected tab	Enforce the current decision through declarative rules and site guards. Added targets enter active policy on save; removed targets stay until 4 AM. Full live matrix remains required.
AC-03f	Private mode / another profile	Protected only where loaded and enabled. Unconfirmed contexts must not be advertised as covered.

F06 — Explain the block and return safely

Trigger: navigation to a selected site while proof is absent, incomplete or expired. **Output:** remaining habits, access reason, habit date/sync context and useful next actions.

1. **Open Habitify** opens the installed Habitify app when available, otherwise its web app. It does not complete a habit inside Habit Gate.
2. **Sync now** requests refresh. It is subject to backoff and asynchronous completion; cached response does not mean a fresh check succeeded.
3. **Go back** offers an exit from the interruption. Verify history behavior, including direct entry with no useful prior page.
4. **Continue to website** is offered only when proof allows access and the requested HTTP(S) destination passes validation.

AC-06a: partial/unknown/expired states never offer a usable Continue. **AC-06b:** reject non-web URLs, embedded credentials and unrelated hosts as return targets. **AC-06c:** a destination becoming invalid or proof expiring before the click cannot grant access. **AC-06d:** keyboard focus, narrow widths and long habit names remain understandable.

Failure contract: a missing native host or API error explains the issue and retains only valid prior proof. Define destination minimization/retention before distribution; the current blocked-page URL can carry the original path/query. [E02, E07]

2.3 Feature specification: daily feedback and Mac behavior

F05 — Show progress and connection health

Surfaces: menu-bar title, Today panel and Overview. **Input:** current snapshot, credential/sync state and browser heartbeat. **Output:** blocked/allowed status, due/completed/remaining counts, active habit date, last successful sync and coverage context. **AC-05a:** counts include only due habits; not-due rows remain visible as rest information. Configuration problems remain blocked. **AC-05b:** "Syncing" is distinct from "Synced at" and an error never updates the last-success timestamp. **AC-05c:** expired heartbeats disappear; permission/private-mode changes remain visible despite ignored heartbeat-only timestamp updates. **AC-05d:** users can identify status, next action and sync health without interpreting colors alone. Profile-specific coverage wording is a proposed release refinement.

F07 — Guard selected Mac apps

Precondition: an active app list and blocked/expired proof. **Input:** up to 50 valid bundle IDs, excluding Habitify, Habit Gate and protected system controls. **Trigger:** launch/activate/unhide events or foreground fallback. **Action:** hide selected windows, preserve the process and show the habit notice for a foreground attempt. Save apps guards additions immediately; removals remain guarded until 4 AM. Habit edits remain independent and deferred.

Criterion	Expected result
AC-07a	Exact selected bundle ID is guarded; an unrelated app is unaffected. No name-based fuzzy matching.
AC-07b	A newly added running app is hidden immediately while blocked; a removed app stays guarded until 4 AM. Re-lock hides selected apps without stealing focus for a background hide.
AC-07c	If macOS refuses hiding, bounded retries end in an actionable protection issue. No forced quit or false success.
AC-07d	Valid completion permits opening/Continue. Repeated activation while blocked is intercepted; sessions and drafts remain intact.

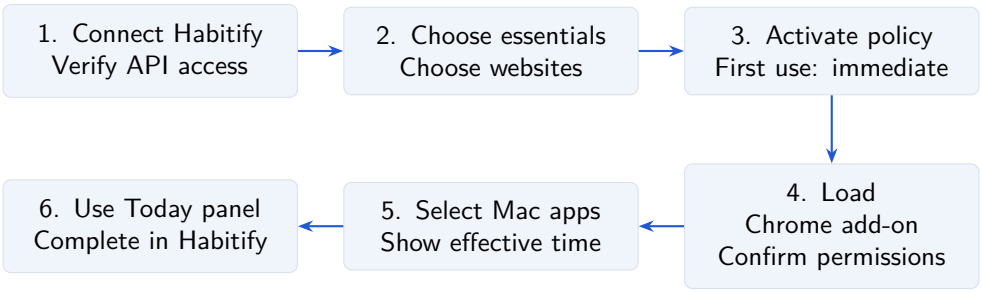
Boundary: background activity, notifications, process execution and other accounts are not denied. Quitting Habit Gate bypasses this voluntary guard.

F08 — Login launch and optional notification

Input: independent login-launch and unlock-notification preferences. **Flow:** register/unregister through macOS ServiceManagement; request notification permission when enabled. Preferences are not an access bypass and do not alter the active daily policy. **AC-08a:** enabled login launch is verified by a real login on a test account, not only a toggle. **AC-08b:** registration failure is explained. **AC-08c:** denied notification permission does not affect sync or blocking. **AC-08d:** closing windows leaves menu-bar operation active. Optional unlock notification occurs on the verified blocked-to-allowed transition, not each poll. [E01, E06, E08]

3 User journeys and application flow

First-time setup



Current friction: the Chrome step requires Developer mode and a local folder. New app selections apply when saved, including after initial activation. Chrome custom sites need a permission grant and normally refresh within 30 seconds. **Proposed:** one setup checklist with explicit "active now" and "starts at [time]" results; distinguish selection from verified coverage.

The daily loop

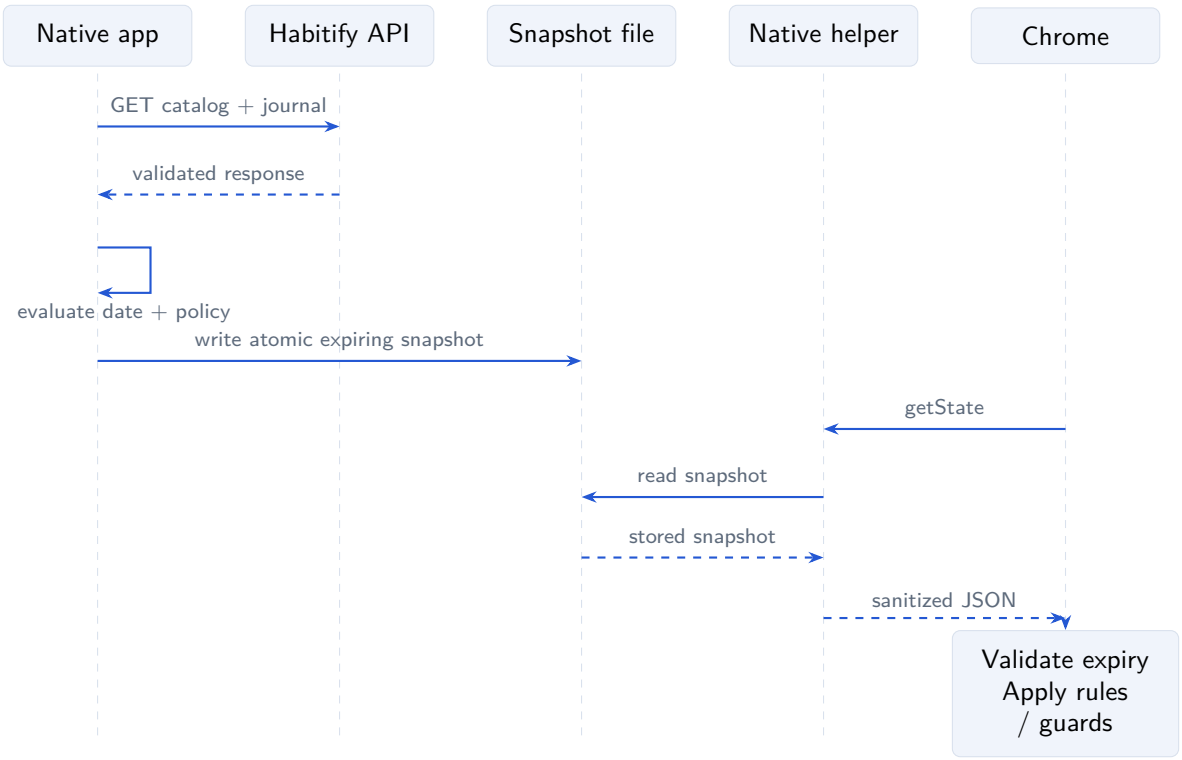
1. Launch at login if enabled. Initialize cached rules, the app guard and housekeeping before asynchronous Keychain retrieval.
2. Resolve the local habit date. Fetch the catalog and journal every 60 seconds while awake, subject to backoff, and on launch, wake, network recovery or manual refresh.
3. Evaluate all selected scheduled habits. Persist an expiring snapshot for browser adapters. Render the same remaining essentials in native views.
4. A selected site opens the blocked page; a selected Mac app is hidden and a native notice appears. The next meaningful action is to complete the habit in Habitify.
5. When every due essential is verified complete, allow access until 4 AM. A successful later fetch showing an undo re-locks access. [E05–E08]

Exceptions that must remain understandable

Situation	User flow
Keychain waiting	Keep the interface responsive and enforce cached rules. Ask the user to approve the macOS prompt locally.
Offline / API error	Show the last successful sync and reason. Preserve only a still-valid prior unlock; otherwise stay blocked. Retry automatically.
Settings changed	Apply added websites/apps now; defer removals and habit edits to 4 AM. Each screen saves only its own field; preserve other pending edits and show the boundary.
Missing habit	Explain the configuration problem. Help select supported habits for the next day; do not silently classify it as rest.
Browser missing	Explain that the Chrome profile is unconfirmed; app running does not mean every browser is protected.

3.1 Sequence diagram: verification to browser enforcement

CURRENT SUCCESS PATH / FAILURE BEHAVIOR BELOW



Where the user fits

The user records progress in Habitify. Habit Gate reads that cloud data; it does not write completion. The app normally attempts verification every 60 seconds. Chrome normally reads local native state on a 30-second alarm. Cloud propagation, network time, housekeeping and backoff can add delay. Do not promise instantaneous or strictly 60-second end-to-end unlocking; measure the actual journey during the pilot.

Failure and boundary branches

Branch	Required behavior
API request fails	Keep the last successful timestamp, attach a safe error, retain only unexpired verified proof and schedule backoff.
Date/policy changes in flight	Discard stale results and verify the current date/policy. Yesterday's response cannot restore access.
Snapshot write fails	Surface the failure. Browser may retain older state; resolve the durability risk in R01 before unsupervised alpha.
Native host unavailable	Chrome keeps only a valid cached proof, applies expiry locally and reports the missing connection.
Manual refresh	Helper returns cached state after posting a request. UI must observe a new verified result; acknowledgement alone is not success.

[E05–E09]

4 Business rules and daily time model

The unlock rule

Allow access only after a successful verification finds no configuration problem and every selected habit scheduled for the active habit date has status `completed`. A numeric progress total alone never unlocks access.

Habit result	Gate	Reason
All due habits completed	Allow	Valid proof lasts until its local 4 AM expiry.
Any partial / inprogress	Block	Completion status is the authority.
Any skipped / failed	Block	These statuses do not satisfy an essential.
Unknown / absent journal	Block	Completion is not verified.
Missing / archived / ended	Block	This is a configuration problem.
Unsupported weekly goal	Block	Not a valid first-version selection.
All selected habits not due	Allow	Only after validating every selected habit and its schedule.
Successful fetch shows undo	Re-lock	Prior completion is superseded by current verification.

4 AM is the companion's boundary

Examples use local UTC+07:00. Boundaries use calendar arithmetic, so daylight-saving days elsewhere may last 23 or 25 hours.

Mac time	Journal date	Consequence
18 Oct 2026, 00:30	2026-10-17	Late-night check-ins belong to the previous habit date.
18 Oct 2026, 03:59:59	2026-10-17	Yesterday's valid proof may still apply.
18 Oct 2026, 04:00:00	2026-10-18	Yesterday's proof expires; today's date needs verification.
18 Oct 2026, 16:00	2026-10-18	Complete today's scheduled essentials.

Habitify itself uses a midnight day boundary. Habit Gate does not change Habitify's reset. This mismatch must be explained in onboarding and beside the displayed habit date.

Policy changes and offline proof

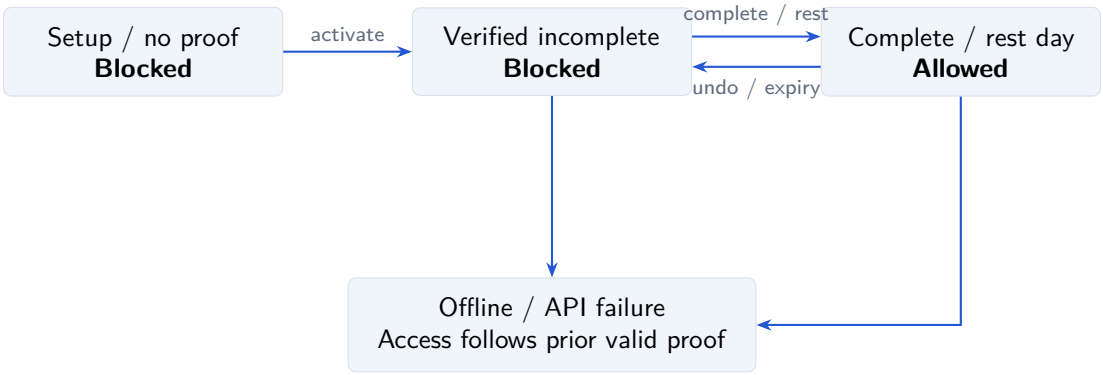
First activation is immediate. Later saves follow this table. Each screen saves only its field; Save websites cannot commit an unsaved habit draft.

Saved change	Effective behavior
Habit addition or removal	Next local 4 AM; today's essentials stay unchanged.
Website or app addition	Active on save. Chrome normally refreshes within 30 seconds; custom domains need permission.
Website or app removal	Remains protected until the next local 4 AM.

Mixed saves add now and remove at 4 AM. Re-adding cancels removal; removing a new target still waits. Target-only edits preserve valid proof. Offline access ends at expiry; missing proof blocks. [E11]

5 State model and interaction semantics

The access decision has two values: `blocked` or `allowed`. Setup, syncing, connection failures and configuration warnings are explanatory UI states around that decision. "Syncing" must never itself imply permission.



Visible state	Explain	Offer
Setup incomplete	Connection or policy is missing.	Connect, choose essentials, activate, verify browser.
Blocked	Number remaining and habit date.	Open Habitify; Sync now.
Sync in progress	Verification is underway; existing access decision remains in force.	Disable duplicate refresh while native sync runs.
Allowed	Complete or rest day, with expiry.	Continue to requested site/app.
Offline with proof	Last successful sync and expiry.	Retry; show why access is temporarily retained.
Offline without proof	Verification is unavailable.	Retry; keep blocked.
Configuration issue	Name the invalid/missing selection.	Open habits; explain that habit edits wait until 4 AM.
Browser coverage issue	Profile heartbeat, permissions or private mode is missing.	Installation / permission guidance.

Asynchronous refresh must be explicit

The native bridge's `refresh` command posts a refresh notification, then returns a cached snapshot. It acknowledges a request; it does not wait for the Habitify request to finish. The browser UI must wait for a newer verified timestamp before presenting a fresh success or Continue action. The existing date/expiry checks still govern access. [E06–E08]

Proposed UX acceptance: a user can answer "Am I blocked, what remains, and is sync working?" without opening settings. Never show an unlocked status based only on a successful bridge connection or an accepted refresh request.

6 Interface review and screenshot gallery

6.1 Overview: answer the daily questions

CURRENT APP / CAPTURED 18 OCTOBER 2026

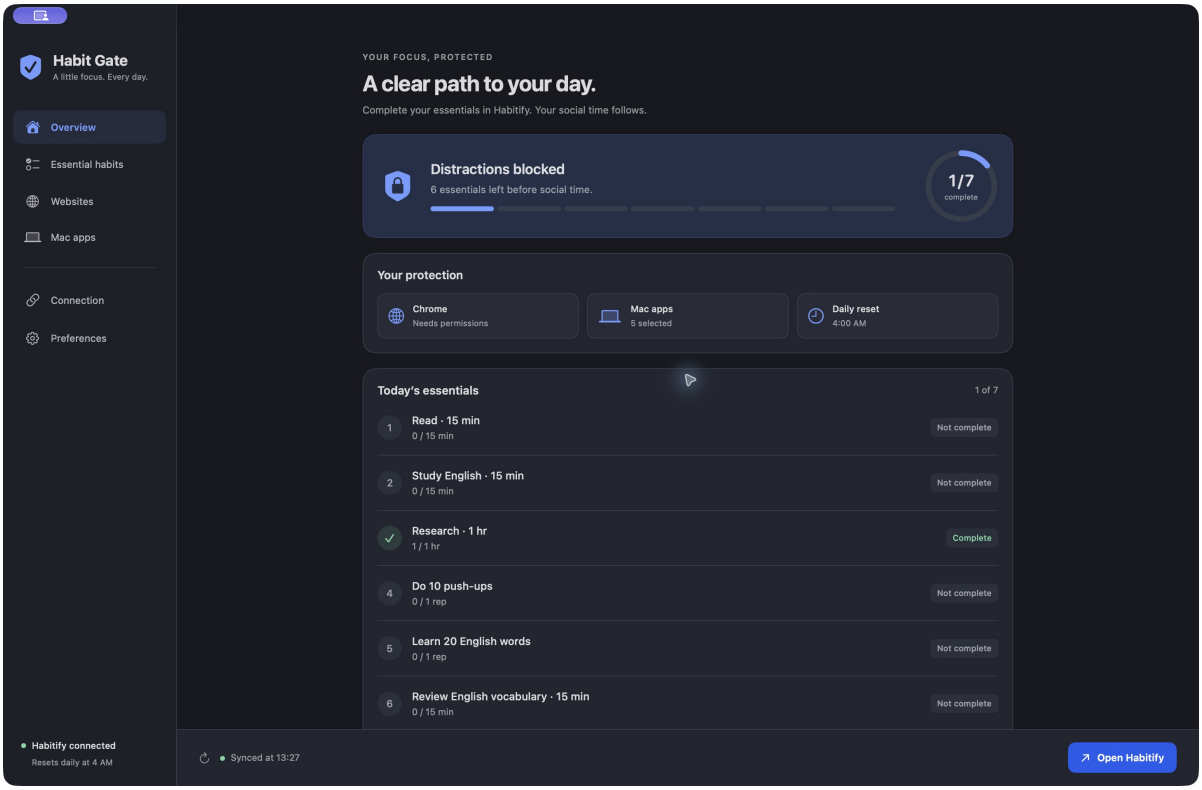


Figure 1: *Installed v1.3.0 Overview with real personal example habits. A blocked state and the current Chrome permission status are visible. This capture is not an unlock test.*

Purpose. Give a clear daily answer before asking the user to navigate: access status, progress, remaining essentials, habit date and sync health.

- **Works well:** calm hierarchy, readable habit names, explicit remaining count, protection cards and an accessible Open Habitify action.
- **Improve before alpha:** distinguish "five selected apps" from verified enforcement health; distinguish one connected Chrome profile from coverage of every profile.
- **Acceptance:** five pilot users can correctly explain status, remaining action and sync time without instruction. Record task outcomes; do not infer usability from this screenshot.

[S01; E01, E08]

6.2 Essential habits: choose a realistic commitment

CURRENT APP / ACTIVE AND DRAFT CHOICES

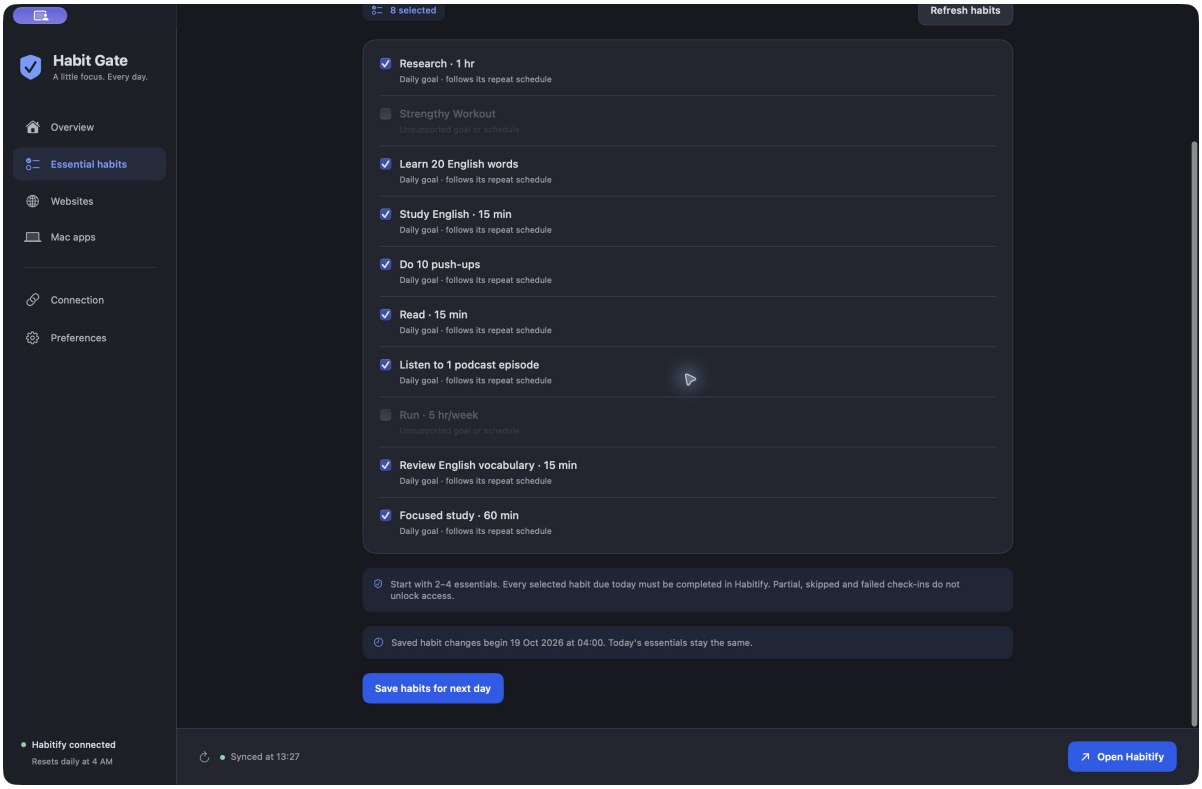


Figure 2: *Installed habit selector, scrolled to the timing notice and Save action. Weekly or unsupported goals are disabled. Eight essentials are selected for the next boundary; seven are active today. Selecting 2–4 remains the onboarding recommendation.*

Purpose. Choose the habits that determine access without editing Habitify itself. The catalog uses real Habitify identifiers, schedules and goal types.

- **Works well:** unsupported selections explain why they cannot be used; the habit-only next-day notice prevents surprise changes to today's essentials.
- **Improve before alpha:** label the draft and active selection separately, show the exact effective date/time beside Save, and give missing habits a direct recovery explanation.
- **Acceptance:** editing a selection after activation does not change today's rule. Relaunching preserves both active and pending policy. Unsupported or missing selections cannot silently produce a rest-day unlock.

[S02; E05, E08]

6.3 Websites: make coverage visible

CURRENT APP / CHROME-FIRST SCOPE

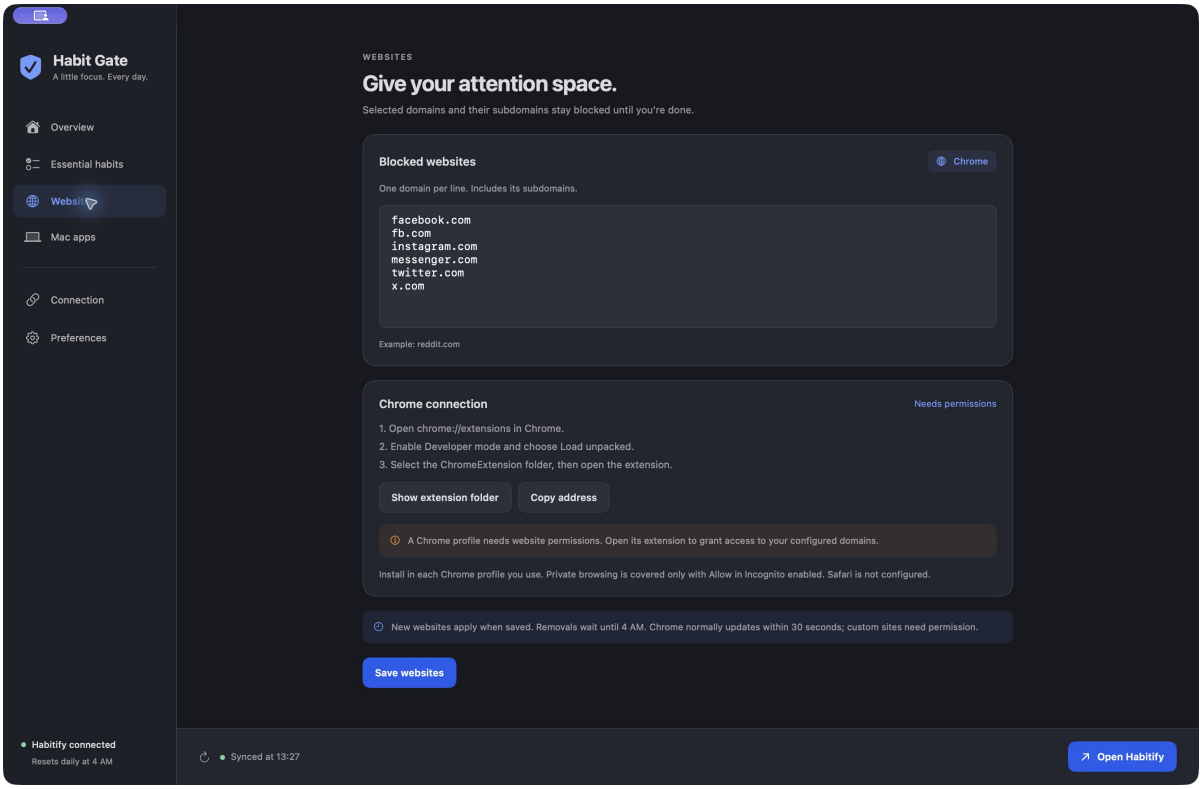


Figure 3: *Installed website settings. Domains cover their subdomains. Save websites applies additions now and removals at 4 AM. The setup instructions still require an unpacked Chrome extension.*

Purpose. Maintain a small domain list and explain exactly which browser context is covered.

- **Works well:** the domain format is simple, installation steps are visible, and Safari/private-mode limitations are stated.
- **Improve before alpha:** replace the single Connected label with profile-specific permission and private-mode status, a last-seen time and a clear test action. Do not ask for more permissions than the selected domains need.
- **Acceptance:** `m.facebook.com` matches Facebook; `notfacebook.com` does not. A missing custom-domain permission produces an explicit incomplete-coverage state.

[S03; E07, E08]

6.4 Mac apps: be honest about the guard

CURRENT APP / WINDOW HIDING, SESSIONS PRESERVED

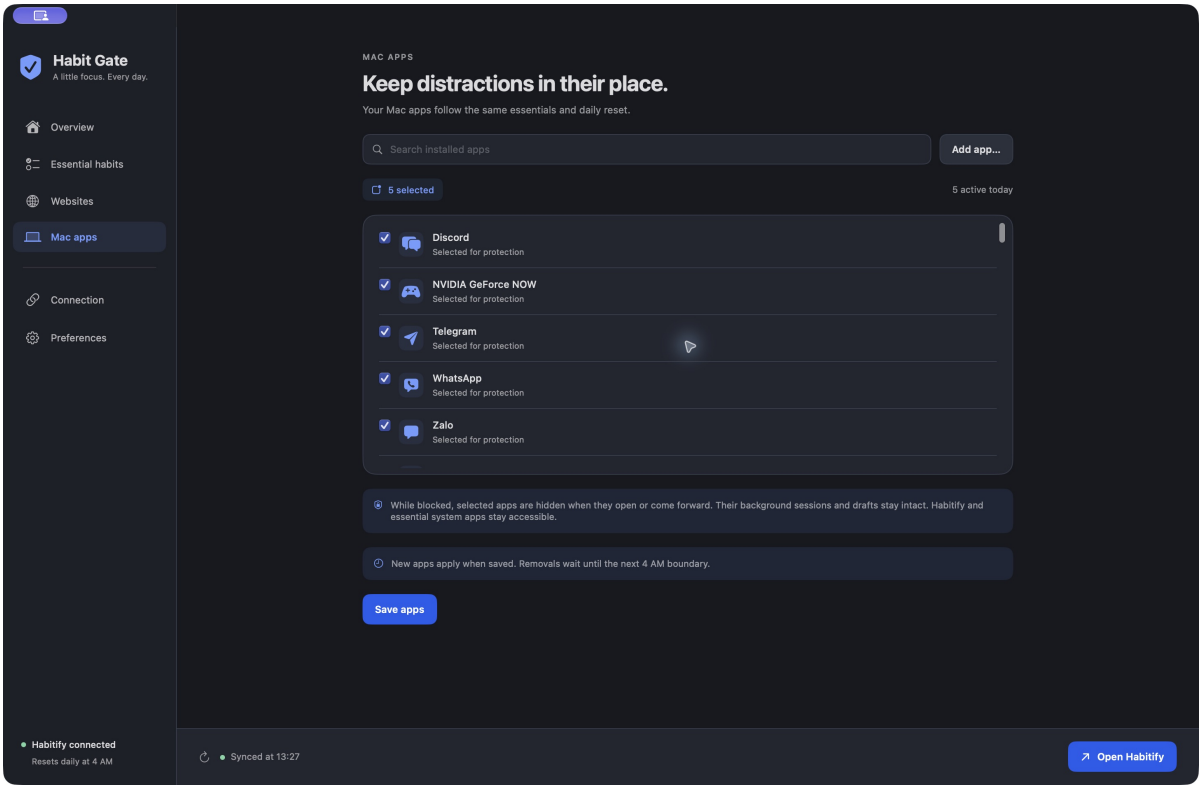


Figure 4: *Installed app selector with Discord, NVIDIA GeForce NOW, Telegram, WhatsApp and Zalo selected. The exact bundle identifier determines matching. New apps apply on save; removals wait until 4 AM.*

Purpose. Apply the same habit gate to foreground distractions while avoiding forced termination and lost work.

- **Works well:** selected apps appear first, search is available, and the explanation preserves the user's mental model of background sessions.
- **Improve before alpha:** call this a "foreground app guard" in trust-sensitive copy. Show hide failures distinctly from a successful selection. Review whether any essential communication apps need a clearer suitability warning during selection.
- **Acceptance:** an unrelated app remains usable, a selected foreground app is hidden while blocked, drafts remain intact, and a refused hide is reported rather than presented as success.

[S04; E06, E08]

6.5 Connection: reduce setup uncertainty

CURRENT APP / NO KEY EXPOSED IN THIS CAPTURE

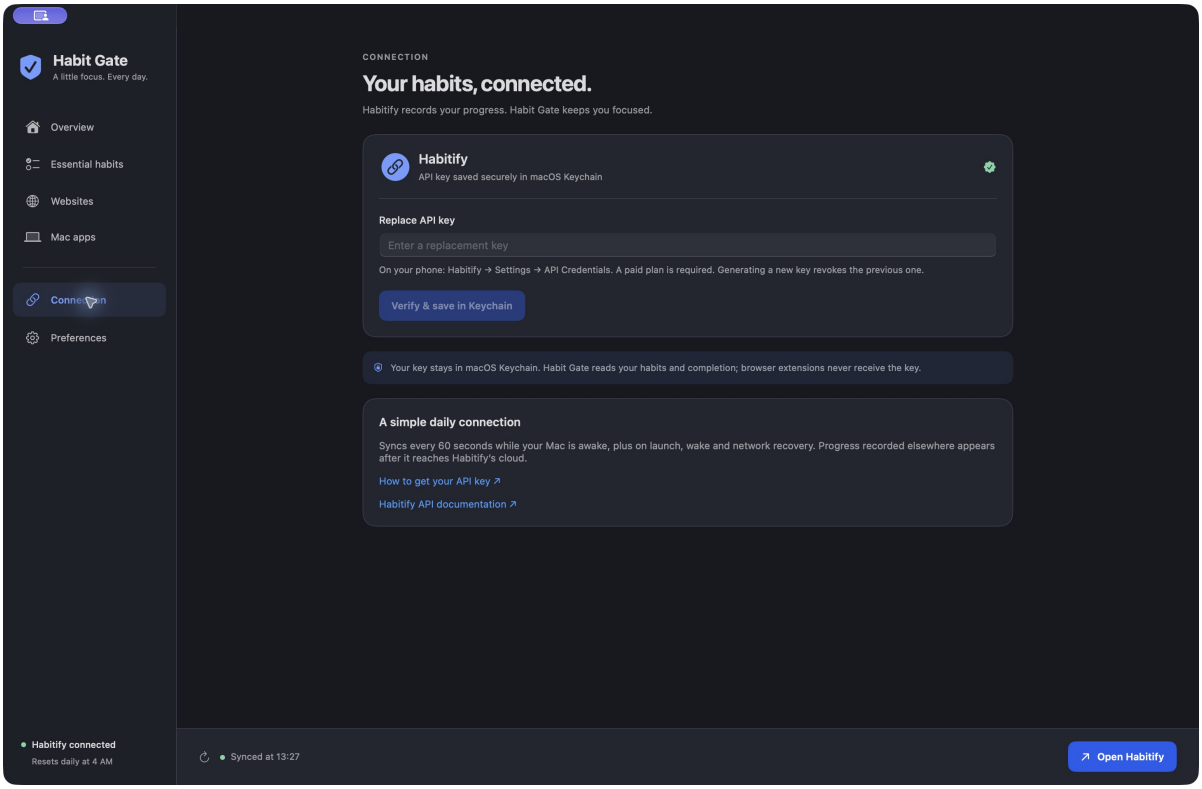


Figure 5: *Installed connection screen. The replacement field is empty. The saved credential is in macOS Keychain and is not displayed or included in this document.*

Purpose. Connect an existing Habitify account with the minimum required explanation and a trustworthy recovery path.

- **Works well:** paid access and mobile API-key creation are explained; verification precedes saving; browser adapters never receive the credential.
- **Improve before alpha:** verify current provider eligibility and terms, clarify that the app uses read-only requests even if the credential itself is broader, and provide a local disconnect/delete-key journey. Disconnect is proposed, not currently implemented.
- **Acceptance:** a denied Keychain prompt leaves UI and protection responsive. Reconnecting never writes habit progress. Errors and support exports exclude the submitted key.

[S05; E06, E09; O01–O02]

6.6 Preferences: show the daily contract

CURRENT APP / LOGIN LAUNCH ENABLED

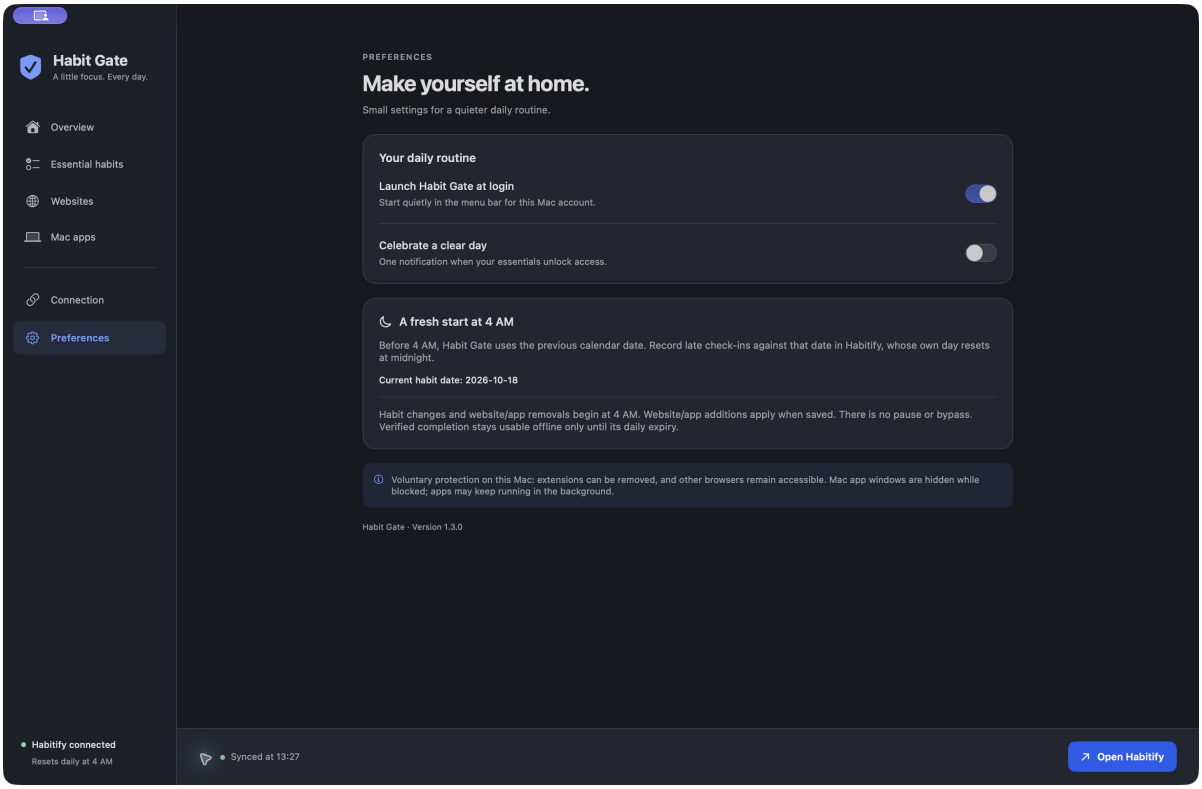


Figure 6: *Installed v1.3.0 preferences, with launch at login on and the optional unlock notification off. The habit date and voluntary coverage statement remain visible.*

Purpose. Keep recurring behavior predictable: quiet startup, an optional unlock notification and a clear 4 AM explanation.

- **Works well:** the midnight versus 4 AM distinction is explicit, version information is available and the product does not imply tamper resistance.
- **Improve before alpha:** show system login-registration failures as actionable results. Add version/update and support information once public distribution exists.
- **Acceptance:** restart/login verifies registration on a clean test account. A denied notification permission does not affect blocking. Turning notifications on is independent of policy activation.

[S06; E01, E08]

6.7 Today panel and native interruption

LEFT: ILLUSTRATIVE FIXTURE / RIGHT: CURRENT APP

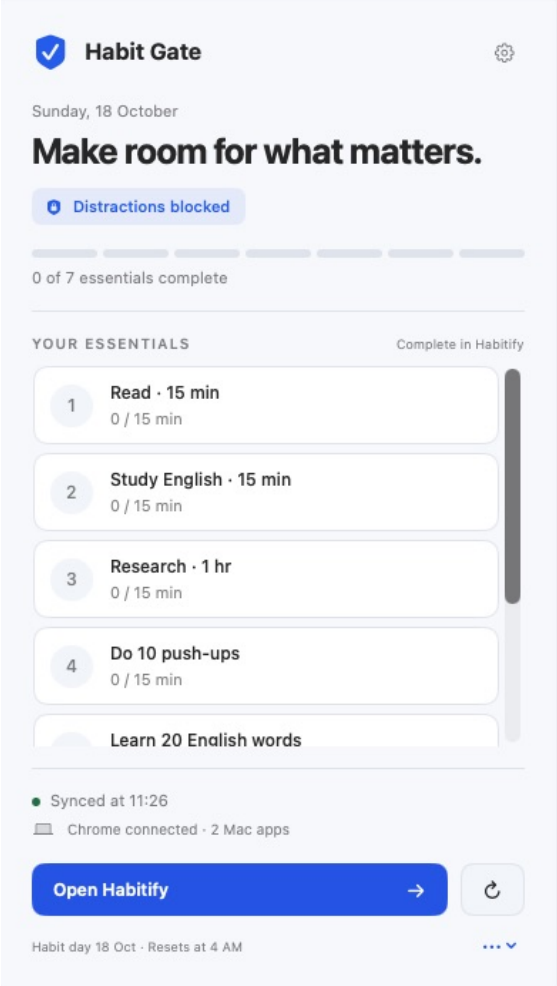


Figure 7: *Today panel rendered by an in-memory UI fixture. Preview canvas is trimmed for legibility. Progress and coverage values are illustrative.*

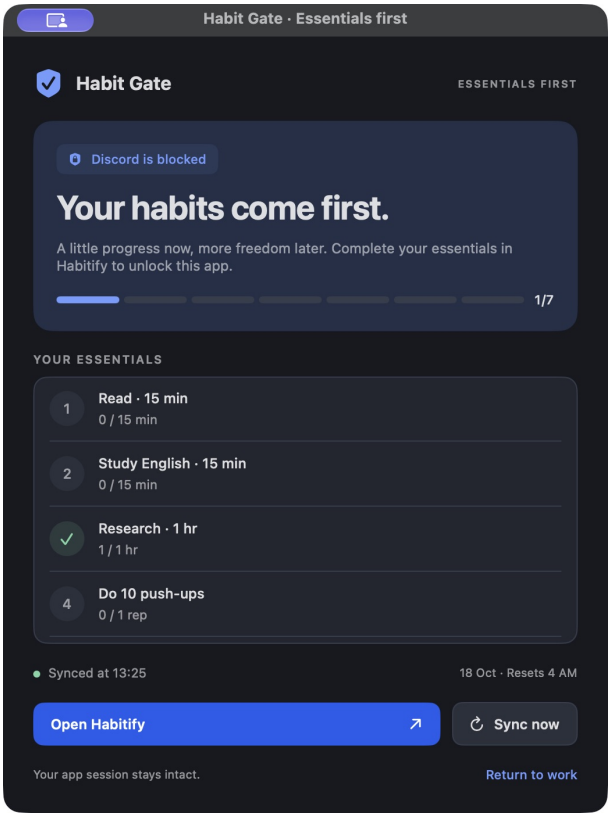


Figure 8: *Real installed blocked-app notice for Discord. The app explains the remaining habits and preserves the session.*

Interaction goal. The menu-bar panel should provide a fast answer, while the interruption should provide a useful next action. Neither should replicate Habitify's history grid.

- Keep text alongside colors, visible keyboard focus and scrollable long lists. Expose habit names and status to assistive technologies.
- Show Continue only when the same valid proof used by enforcement allows it. The native notice otherwise offers Open Habitify, Sync now and Return to work.
- **Proposed check:** test with VoiceOver, keyboard only, increased text size, reduced motion, long localized names and light/dark appearance. Existing fixture review is useful but is not a full accessibility audit.

[S07–S08; E02, E08]

6.8 Browser interruption and extension popup

RENDERED UI PREVIEWS / MOCK EXTENSION MESSAGING

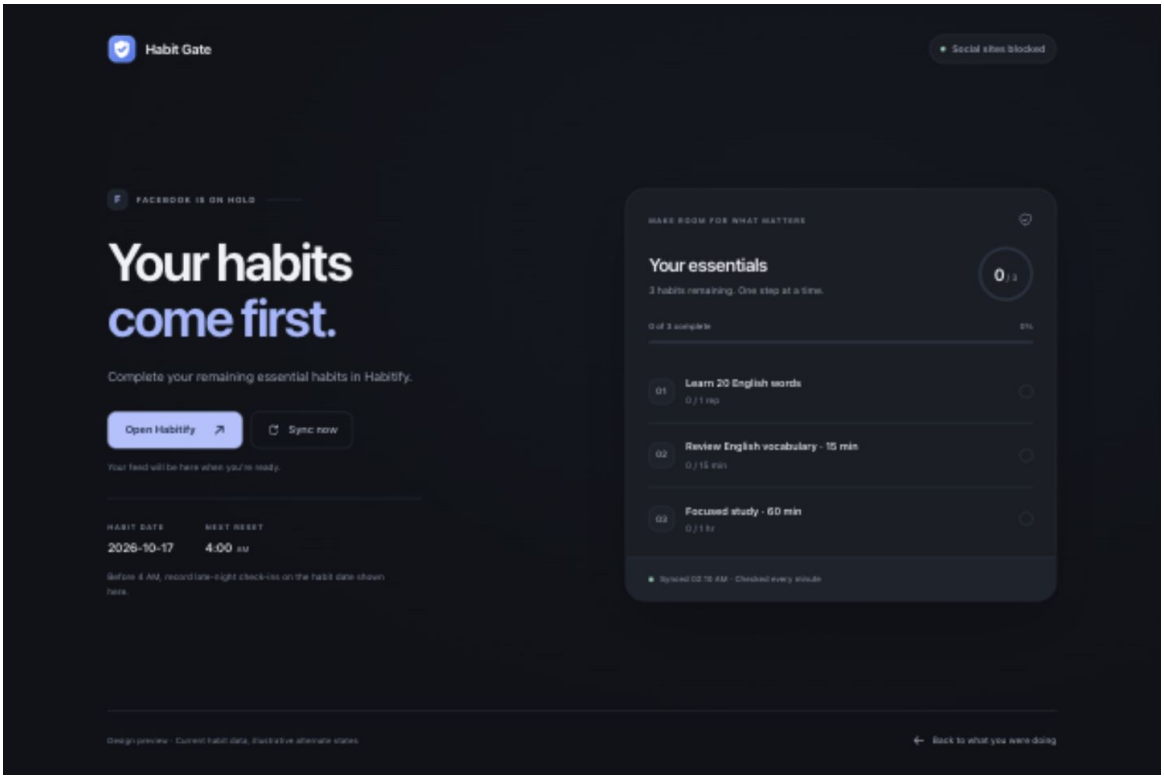


Figure 9: *Existing local preview of the implemented Chrome blocked page. Habit data and bridge responses were mocked for visual review. The user previously confirmed the installed design appeared in Chrome.*



Figure 10: *Popup preview with the same visual language; illustrative progress.*

Works well. A clear headline, remaining-habit card, progress ring, local assets and a calm next action. The responsive preview was reviewed at desktop and narrow widths.

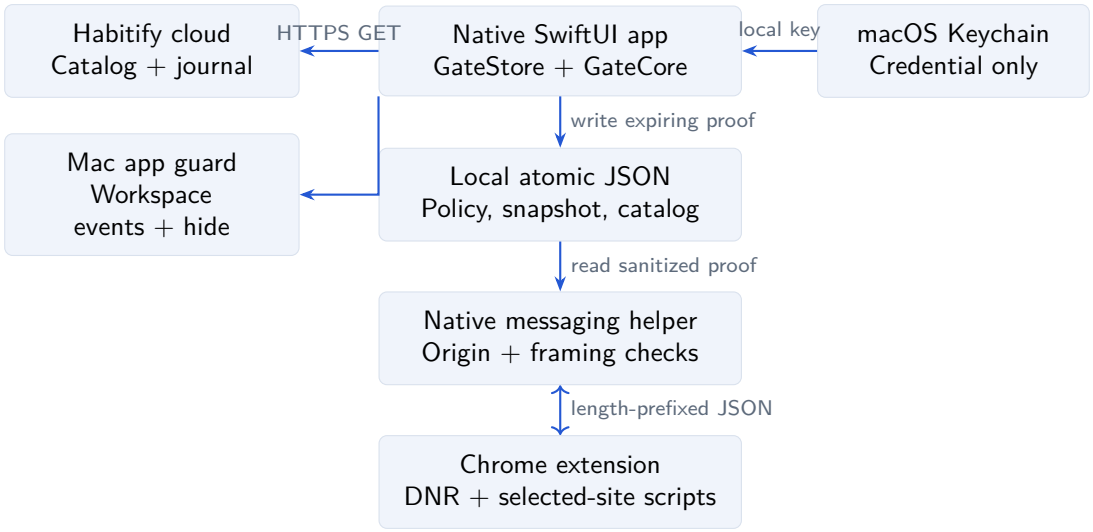
Review next. Make connection and permission failures actionable without exposing technical internals. Validate browser Back behavior and expired Continue destinations. A cached destination may include sensitive query parameters; define its retention before public release.

Acceptance. Direct navigation, redirects, subdomains and already-open tabs block; unrelated sites remain usable. Existing automated adapter tests pass, but the full live navigation matrix is still a release gate.

[S09–S10; E02, E07]

7 Architecture and responsibility boundaries

CURRENT BASELINE / THREE LOCAL COMPONENTS



Component	Owns / must not own
`GateCore`	Calendar dates, schedules, domains, app policy and proof validation. No UI or network dependency.
Native app	Habitify requests, Keychain session, active/pending configuration, presentation, login and notifications. Main-actor state; asynchronous I/O.
`AppBlocker`	Activation/unhide/launch observers, session-preserving hide requests, bounded retries and issue reporting. No habit editing.
Native helper	Fixed extension-origin validation, bounded input, sanitized cached response and permitted refresh/open commands. No API key access.
Chrome adapter	Declarative website rules, site guards, expiry, coverage heartbeat and blocked UI. No credential or app-selection data.

Scheduling today. One five-second housekeeping loop handles date/proof checks, browser heartbeat reads and the foreground fallback. Habitify sync is due every 60 seconds; Chrome requests local native state on a 30-second alarm. Those local reads are not 30-second Habitify API requests. Selected-site scripts also check expiry every second. Keep this simple baseline unless profiling identifies a material issue. [E05–E08]

8 Data model, interfaces and invariants

Contract	Essential fields and invariant
Configuration	Active policy includes saved target additions; optional pending desired policy retains habit edits/removals with effective time/timezone; notification preference. Policy contains habit IDs, domains and app bundle IDs.
`GateSnapshot` v1	Habit date, timezone, UTC offset, expiry, access, reason, habit states, domains, last successful sync and optional sanitized error. No API key or app list.
Browser client	Random local profile ID, last seen, private-access flag and permission result. A heartbeat is live for less than 150 seconds; over-five-second future skew is rejected.
Keychain item	Generic password under the app's service/account identity. Only native credential code reads/saves it. The key can remain in native memory for requests.

[E05–E07, E09]

Habitify adapter

```
GET https://api.habitify.me/v2/habits?limit=100&offset=...
GET https://api.habitify.me/v2/habits/journal?date=YYYY-MM-DD
Header: X-API-Key: [local credential; never logged]
```

The catalog handles offset pagination with duplicate checks and a 100-page bound. Each request has a 20-second timeout. HTTPS redirects are rejected rather than forwarding credentials. Authentication, rate limits, malformed data and server failures produce sanitized errors. A response for a previous habit date or superseded active policy is discarded. Re-check provider access, schema and API terms before alpha; no API availability SLA has been established.

Local browser protocol

Chrome launches the registered helper and exchanges a four-byte little-endian length followed by JSON. The helper accepts input up to 65,536 bytes, validates the fixed extension origin and permits only `getState`, `refresh` and `openHabitify`. Content scripts can request gate status; privileged messages must originate from extension pages.

```
Request:  {"command":"getState","clientID":"local-random-id",
           "privateAccess":false,"permissionsOK":true}
Response: {"snapshot":{...version 1, expiring proof...},
           "configured":true,"hostAvailable":true}
```

Versioning gap. The native app is 1.3.0, the extension manifest is 1.0.0 and the snapshot schema is v1. These are separate version domains; formalize a compatibility matrix and reject unsupported future schemas visibly.

Persistence and compatibility

JSON writes are atomic and owner-only: directories 0700, files 0600. Existing policies without app selections decode as an empty app list. Atomicity avoids partial JSON; it does not create a backup, protect against a user editing files, or guarantee synchronized policy/proof commits. Add recovery tests before advertising robust storage. [E05–E07]

9 Software engineering principles

Decision rule

Favor the smallest change that demonstrably improves correctness, usability or operating cost. Add an abstraction when a real responsibility or test boundary requires it, not to satisfy a diagram.

Principle	Applied today	Focused improvement
KISS / YAGNI	Native app, pure core, small helper, no separate Habit Gate backend or external packages.	Keep Chrome-first scope; avoid rewards, ML and microservices until evidence supports them.
Single responsibility	GateCore separates policy; AppBlocker isolates app enforcement; credentials have their own session/vault.	GateStore still coordinates many tasks. Extract only a subsystem with repeated defects or a clear test seam.
Dependency inversion	CredentialStorage and API transport allow controlled tests. App guard accepts fake applications.	Add injectable persistence/clock seams for recovery scenarios; avoid a general plugin framework.
Deterministic domain logic	Date, schedule and access evaluation take explicit time/calendar inputs.	Maintain one set of fixtures shared with JavaScript to catch native/browser drift.
Least privilege	Keychain stays native; browser gets a small expiring snapshot; API behavior is GET-only.	Explain optional host permissions and document the provider key's actual privileges.
Resource lifetime	Closed panels release SwiftUI hosting views; unchanged catalogs and coverage do not republish/write.	Profile meaningful workloads before revising timers or caching. Do not trade correctness for small unproven savings.
Explicit failure	Unknown completion blocks; app-hide failures are surfaced; API errors use backoff.	Surface corrupt policy and failed persistence as first-class recovery states.
Safe evolution	Pending settings prevent immediate weakening; old app-less policies migrate.	Add schema compatibility, tested migrations and a documented rollback package.

Working agreements for maintainers

A change starts with one observable problem and an acceptance example. Review the smallest patch, run the relevant tests, and record what is still unverified. Keep secrets and account data out of fixtures. Treat a successful build, automated tests and live acceptance as different kinds of evidence.

Optimization decision record. An event-driven 1.2.2 experiment was reverted. Short measurements did not establish a clear practical benefit, and the owner preferred simplicity. The 1.3.0 timing update retains the simple scheduler restored in 1.2.1; the abandoned scheduler/watcher refactor is not part of this architecture. Revisit only with a reproducible profile and a measurable user benefit. [E02, E04]

10 Reliability and failure recovery

Failure / transition	Current response	Release acceptance
API 401 / 403	Sanitized reconnect/eligibility error; prior valid proof can survive until expiry.	Revoke a test key; verify UI, backoff, expiry and local reconnection. No key appears in logs.
429 / server / network	Retry at 1, 2, 5, then 15 minutes; honor a longer Retry-After.	Backoff prevents request storms; recovery updates both native and browser state.
Keychain authorization waits	Dedicated actor keeps UI, app guard and housekeeping running.	Controlled tests pass; observe a real authorization wait on a disposable account.
4 AM / wake / timezone	Re-evaluate the active date/proof; wake and time events trigger refresh.	Live exact-boundary, wake and timezone matrix; quantify enforcement delay. Native five-second polling can add seconds.
Stale API response	Discard changed-date or changed-policy results.	In-flight request spanning reset never restores yesterday's unlock.
App hide rejected	Verify after 0.15 seconds, retry twice at 0.25 seconds, then report failure.	Supported apps and clean accounts show an actionable issue; never claim process denial.
Missing browser heartbeat	UI drops clients after 150 seconds. Cached browser proof still expires independently.	Remove permissions / close profile / disable add-on; coverage text stays accurate.
Corrupt policy / disk full	Read failure can fall back to default config; write errors show messages.	**Proposed critical fix:** explicit recovery state, validated last-known policy and persistence-failure tests.

[E02, E05–E09]

Storage risk found by inspection

`GateFiles.read` returns nil for missing, unreadable or malformed JSON. Startup then uses a default configuration. Losing an active custom policy can therefore lose its app list or custom target knowledge. Separately, `persistSnapshot` updates the native app guard before the disk write; a failed write may leave the browser reading an older snapshot. These paths require controlled fault-injection tests and a defined recovery policy. They have not been reproduced against the owner's live files.

Proposed recovery contract: distinguish first launch from damaged existing state, never label default coverage as restored custom coverage, retain validated known targets where possible, and show a recovery-required message until durable state is consistent. Preserve a last-known-good copy with a bounded migration/restore path. This is targeted reliability work, not a reason to add a database or a daemon.

11 Security, privacy and trust boundaries

VOLUNTARY SELF-CONTROL / NOT A SECURITY BOUNDARY

The Mac owner can remove the add-on, use another profile/browser, quit the app or edit same-user files. Local proof is not cryptographically signed against that user. The current product should promise useful friction and clear feedback, not resistance to a determined owner.

Data	Location / destination	Public-release obligation
Habitify API key	Keychain; native memory; HTTPS to Habitify. Never sent to the browser.	Define disconnect/delete-key behavior, key replacement guidance and a no-secret diagnostics policy. GET-only app behavior does not prove a read-only-scoped key.
Habit IDs, names, progress	Local catalog/snapshot; selected habit states reach the local extension.	Explain purpose and local retention. Habit names can reveal sensitive routines; minimize support exports.
Domain / app policy	Local configuration; only domains reach browser proof.	Document reset, backup, migration and uninstall handling.
Blocked destination	Local blocked-page URL, potentially including path/query.	Review browser-history exposure, destination minimization and expiry. Do not include raw URLs in diagnostics.
Profile heartbeat	Local random ID, last seen, permission/private-mode flags.	Explain it is local coverage data; no user identity or analytics service is needed.

Controls already present

HTTPS API calls reject redirects; errors omit response bodies and request headers. Native messaging checks the fixed extension origin, validates command shape and bounds input. The extension uses a local content security policy and limits privileged messages to its pages. Files are owner-only and atomically replaced. The helper receives no key. These controls are implemented and partly covered by automated/installed checks; this is not an independent security audit. [E02, E05–E09]

Focused review before distribution

- Justify each Chrome permission, especially broad optional HTTP(S) host capability used to request individual custom sites. Verify Web Store policy and current provider terms.
- Review untrusted API fields, site destinations and native JSON for injection, bounds and safe rendering. Keep server text and keys out of diagnostics.
- Publish a plain-language privacy notice, permissions explanation, deletion/uninstall steps and vulnerability-reporting contact. Define who handles reports before collecting them.
- Distribute signed/notarized artifacts through an authenticated channel. Establish secure updates and rollback. Do not claim these exist in the ad-hoc build.

Privacy baseline: no Habit Gate analytics/backend or browsing-history upload is implemented. That does not mean Chrome itself keeps no history, or that a future support export is automatically safe. Review a sample export before introducing it.

12 Verification and acceptance evidence

CURRENT RESULT / 46 SWIFT + 8 BROWSER TESTS = 54

Test counts describe executed test functions, not coverage percentages or proof that every real environment works. A controlled fixture, a live observation and a proposed check have different evidential weight.

Requirement	Existing evidence	Remaining acceptance
F01 connection	Mock API auth/pagination/errors; live GET connection; six credential regressions.	Real denied/slow Keychain flow on a clean account; provider eligibility and key revocation journey.
F02/F04 policy	Complete, partial, skipped, failed, unknown, undo, missing/archived, duplicate entry and rest-day fixtures.	Real completion to unlock to undo, with consent using a test account/habit.
F03 websites	Domain/lookalike rules; eight browser tests; live redirects for default sites and a subdomain.	Live redirect chains, already-open pages, permission removal and incognito matrix.
F05 status	Three coverage regressions; real connected profile; screen/fixture reviews.	Profile-specific clarity; five-user comprehension and keyboard/VoiceOver tasks.
F06 interruption	Responsive browser previews; guarded Continue states; real native notice.	Live Back/Continue after expiry, long destinations and narrow displays.
F07 app guard	Fake-app tests and five disposable native integration checks; live Discord interception.	Each supported target app on clean/restarted accounts; refused hide and repeated activation.
F08 preferences	Login enabled; persistence and target-timing tests; isolated website save checked live.	Actual logout/login on a test account; denied notification permissions.
Q01 time/offline	Exact 4 AM, midnight, rollback, timezone and DST calendar tests; mocked proof expiry.	Actual sleep/wake, time changes and in-flight response crossing reset.
Q03/Q04 trust	Installed origin/framing/signature checks; no-key browser response; asynchronous Keychain fixtures.	Corrupt policy/disk-full recovery; security review and sanitized diagnostics.
Q05 resources	Historical 1.2.1 idle (65.02 s) and separate physical footprint.	Long representative resource run, transient helper accounting and energy impact.

[E02–E04, E10–E11]

Minimal public-release test matrix

Test fresh install/upgrade, supported OS and Chrome profiles, accessibility, errors/offline expiry, Keychain denial, pending edits, login/wake, timezone/DST, redirects/open tabs, custom domains and private mode. Record build, environment, expected/observed result and evidence.

Use dedicated test data. Previous verification did not complete or undo the owner's habits.

13 Performance and simplicity budget

Measurement scope. The figures below were measured on 1.2.1 (6), before the 1.3.0 timing update. No new resource profile was run for 1.3.0; these results are not a current-build resource guarantee.

Measurement	Historical 1.2.1 result	Interpretation
Sample duration	65.02 seconds	Warm app; settings sections and blocked notice visited, then panels closed. 63 samples.
Average native CPU	0.154% of one core	Below the 0.5% idle target in this sample. Not a long-duration battery guarantee.
Mean resident memory	107.06 MB RSS	Above the original under-100 MB target.
Peak sampled RSS	120.83 MB	Short sample; no sustained memory bound established.
Separate physical footprint	39.1M; peak 42.8M	macOS app-only accounting, measured separately. Not interchangeable with RSS.
Transient native helper	May be missed	One-second process sampling can miss short-lived Chrome helper CPU and memory.

[E10; copied raw reports in the source package]

Honest conclusion

The measured CPU is small. The resident-memory target remains unmet, and continuous combined-process compliance has not been established. The product can still be useful on this Mac; the CEO should decide whether the original 100 MB RSS requirement is a release gate or a target to revisit. Do not silently replace RSS with physical footprint to claim success.

The software principle: measure before complicating

The retained implementation already releases closed SwiftUI views, avoids a continuously updating relative-time label, uses lightweight system symbols, skips unchanged catalog writes and ignores heartbeat-only UI changes. These are small changes with clear responsibilities.

An experimental scheduler/filesystem-watcher refactor was reverted to preserve simplicity. Its short-run measurements did not establish a clear practical improvement. There is no reason to change language, introduce a daemon or adopt a new framework based on a 0.154% CPU sample alone.

Proposed repeatable measurement protocol

1. Record build, OS, hardware, number of habits/profiles/apps, window state, network and power conditions. Separate cold startup, active interaction and warm idle.
2. After the same navigation routine, close panels and run a representative longer idle session. Observe memory growth, sync counts, energy impact and transient helper peaks; do not compare unrelated short samples as a speedup.
3. Profile only if a regression, battery complaint or growth trend is reproduced. State the specific target and retain the smallest verified fix. Keep correctness and the daily reset ahead of speculative resource savings.

14 Distribution, operations and release gates

Current delivery state

The app is built with Swift Command Line Tools and locally ad-hoc signed. The supplied binary is Apple Silicon; the package declares macOS 13+, but broad OS/architecture compatibility is not established by that declaration. Chrome minimum version is 123 in the manifest. A per-user native host points to the installed app. The extension is unpacked with a fixed identity. Safari is neither packaged nor protected. Before community publication, decide the code/asset licensing model and required notices; no licensing approval is recorded here. [E01, E07–E08]

Gate	Required before	Exit evidence
G01	Unsupervised alpha	Live completion, undo, offline expiry, open tabs, reset/wake and app guard pass on dedicated test data. No unresolved critical correctness issue.
G02	Unsupervised alpha	Damaged policy and failed writes produce defined recovery; active/pending policy survives restart. Native/browser proof decisions agree on shared fixtures.
G03	External alpha	A newcomer can install, connect, confirm coverage and recover without developer help. Explain immediate target additions, next-4-AM removals/habit edits, and private mode. Supervised developer loading is an explicit alpha exception only.
G04	Public release	Developer ID signing/notarization, trusted download, Chrome distribution/permissions review, version compatibility and clean install/upgrade/uninstall validated.
G05	Public release	Privacy and permissions notice, key/data deletion, no-secret support process and security-report contact reviewed. No unsupported tamper-resistance claims.
G06	Public release	Supported-platform matrix, repeatable checks and documented rollback/update path. Resource envelope accepted explicitly, with the current RSS shortfall recorded.

Support and incident handling: proposed minimum

Use a small issue template: app/extension versions, macOS/Chrome version, habit date, access status, sync timestamp, expected behavior and a redacted screenshot. Never request an API key, password, complete browser history or raw sensitive habit list in public issues. Diagnostic export is not implemented; if added, preview/redact before sharing.

For a blocking error: confirm the active habit date and coverage, retry once subject to backoff, inspect sanitized status, and reproduce with dedicated data. Stop a release for incorrect unlock, credential exposure or durable-policy loss. Preserve the last signed package and configuration backup before updates; verify sync, expiry, rules and login registration after rollback.

Release ownership: product approves scope/copy and pilot criteria; engineering owns reproducibility and recovery; a named reviewer signs security/privacy and distribution checks. These are proposed roles, not assigned personnel or completed approvals.

15 Prioritized fixes and risk register

PROPOSED BACKLOG / NO NEW FEATURES IN THIS DOCUMENT

P0 blocks unsupervised alpha. P1 blocks a broad public release. P2 is deferred until the pilot demonstrates a user need. Owners below are proposed roles; effort and dates require estimation after the review.

Priority / ID	Risk or change	Acceptance and proposed owner
P0 / R01	Corrupt policy / failed persistence causes divergent or lost coverage.	Inject malformed JSON, denied writes and disk-full errors. Show recovery; preserve known policy; no stale allowed state crosses a changed boundary. Engineering.
P0 / R02	Critical transitions only have fixture coverage.	Complete G01 using dedicated live data and save build/environment/result evidence. Engineering + QA role.
P0 / R03	Clock/calendar semantics can drift.	Shared fixtures cover midnight, 4 AM, DST, timezones, future timestamps, rollback and Gregorian API dates under non-Gregorian system calendars. Resolve intended differences. Engineering.
P1 / R04	Developer installation and updates create friction.	Clean-account install/upgrade/rollback with signed artifacts, stable extension identity and no hidden manual dependency. Engineering + release role.
P1 / R05	User overestimates browser/app coverage.	Per-profile permissions/private mode and foreground-guard limitation are clear. Pilot users correctly describe coverage and next action. Product + design.
P1 / R06	No defined disconnect, retention or diagnostics policy.	Document and test deleting credentials/data, destination handling and redacted support. Privacy/security reviewer + engineering.
P1 / R07	Provider access or terms change.	Confirm eligibility, endpoint contract, distribution terms and safe key replacement before recruiting users. Product + engineering.
P1 / R08	Resource promise exceeds evidence.	Longer repeatable profile; CEO accepts a measured envelope or funds a specific fix. No under-100 MB claim until demonstrated. Product + engineering.
P1 / R09	Accessibility and platform checks incomplete.	Keyboard/VoiceOver, long names, appearance and supported OS/Chrome matrix pass; publish tested scope. QA + design.
P2 / R10	Scope expansion before demand.	Safari only after signing/toolchain prerequisites and proven demand. Defer exercises, rewards, feeds and analytics. Product.

A simple development sequence

First: stabilize. Address R01–R03 and freeze the daily contract. **Then: pilot.** Run a proposed 5–10 person, consent-based private alpha; measure setup completion, comprehension, incorrect blocks/unlocks and support burden. Do not add telemetry by default. **Finally: distribute.** Complete G04–G06, publish tested limitations and expand only when the pilot supports it. No delivery dates or effort estimates are committed here.

16 CEO review and decision worksheet

Decisions requested

1. **Position:** voluntary adult self-control for existing Habitify users; Chrome websites and a foreground Mac app guard.
2. **Scope:** preserve the daily contract; improve reliability, setup and trust before adding features.
3. **Release:** harden first, then private alpha after P0; public release requires G01–G06.
4. **Resources:** retain the 100 MB RSS gate or explicitly revise it using measured evidence.
5. **Owners:** assign product, engineering and release/security accountability, then estimate budget and dates.

Questions for the review meeting

- Is the paid Habitify/API prerequisite acceptable, and is window hiding sufficient for the intended audience?
- Does the strict no-pause contract support the pilot's needs, and can users explain coverage before activation?
- What single pilot outcome demonstrates value, which platforms are supported, and who handles provider outages?

Proposed decision

Approve targeted hardening and a private-alpha plan. Do not approve broad public release yet. Review the acceptance evidence, onboarding results and measured resource envelope at the next gate.

Record the outcome

Decision	Approve hardening / request changes / defer (select in review)
Product owner	.....
Engineering owner	.....
Conditions / scope	.....
Next review date	.....
Reviewer / date	.....

This is a static review worksheet, not a recorded approval or an interactive form. No CEO decision has been made by this document.

A Evidence index and provenance

The review is based on the local 1.3.0 (8) source, installed-app observations and existing validation artifacts as of 18 October 2026. Source paths below are relative to the separate HabitGate app source folder. Copied reports and screenshot hashes are in this document project's `evidence/` directory. The provenance JSON records SHA-256 hashes for the reviewed source baseline; it is an integrity reference, not a security certification.

ID	Source / what it supports
E01	`README.md`. Setup, daily rules, scope, architecture and operating prerequisites. Copied into `evidence/`.
E02	`VALIDATION.md`. Historical and current build/test/live results, limitations and the optimization rollback. Copied into `evidence/`.
E03	`Tests/GateCoreTests/`, `Tests/HabitGateTests/` and `tests/*.test.mjs`. 46 native and eight browser tests passed for 1.3.0; current counts recorded in E11. E04 preserves the earlier rollback record.
E04	`validation/optimization-rollback.json`. Restored signed 1.2.1 binary, preserved configuration, live sync and regression counts. Copied report.
E05	`Sources/GateCore/`. PolicyEngine, Models, Storage and AppRules: schedules, date/proof, persistence and matching.
E06	`Sources/HabitGate/GateStore.swift`, `Credentials.swift`, `AppBlocker.swift` in that source directory. Sync, Keychain concurrency, fallback and app enforcement.
E07	`Sources/HabitGateBridge/main.swift` and `extension/`: manifest, background, policy and site guard. Protocol, permissions and browser enforcement.
E08	`Sources/HabitGate/App.swift`, `Views.swift`, `DesignSystem.swift`, `MacAppsView.swift` in that directory. Navigation, view lifetime, native appearance and interactions.
E09	`Sources/HabitGate/HabitifyClient.swift`. GET requests, redirect policy, errors, pagination and Keychain implementation.
E10	`validation/reliability-idle-profile.json`, `reliability-final-checks.json` in the validation directory; `scripts/profile.py`. Historical 1.2.1 resource readings and measurement method; not a 1.3.0 profile. Copied reports.
E11	`validation/target-timing-update.json`; `Tests/GateCoreTests/TargetTimingTests.swift` and app-guard tests. 1.3.0 timing, migration, field isolation, installed status and limits. Copied report.

Screenshot provenance

S01–S06 are fresh native captures in order: Overview, Essential habits, Websites, Mac apps, Connection and Preferences. S07 is the existing Today fixture; S08 is the fresh native block notice. S09–S10 are existing local browser UI previews. Each caption states whether the data is actual or illustrative. No screenshot includes an API key. Personal habit names are included for internal review; replace them with synthetic examples before public publication.

B External references and terminology

Official references used by the project

O01 Habitify API documentation.
https://api-docs.habitify.me/

O02 Habitify, “Use Habitify API V2” (mobile API credential guidance).
https://intercom.help/habitify-app/en/articles/6113937-use-habitify-api-v2

O03 Chrome, Declarative Net Request API.
https://developer.chrome.com/docs/extensions/reference/api/declarativeNetRequest

O04 Apple, Running your Safari web extension.
https://developer.apple.com/documentation/safariservices/running-your-safari-web-extension

These links identify the project’s external contracts and installation guidance. They do not establish provider approval, a public distribution agreement or a current API SLA. Re-check eligibility, terms, packaging and permissions against current official documentation before alpha/public distribution.

Terminology for a shared review

Term	Meaning in this review
Active habit date	The local calendar date whose Habitify journal is queried; before 4 AM, it is the previous date.
Essential	A selected supported habit; only scheduled essentials are due on a particular date.
Proof / snapshot	Cached verified state with an expiry, not a cryptographic or tamper-proof certificate.
Rest day	Every selected habit is valid and none is scheduled for the active date. Not a missing-data condition.
Pending policy	Desired next-day policy: habit edits and removals await 4 AM; its target additions are already included in active policy.
Foreground app guard	Selected app windows are hidden; the process, background network and notifications may continue.
DNR / MV3	Chrome’s declarative network rules / Manifest V3 extension model.
RSS	Resident set size: pages resident in RAM. Here reported in decimal MB by the profiler.
Physical footprint	A separate macOS memory-accounting measure; it cannot substitute for the RSS target.
Private alpha	Small controlled user group, explicit limitations and support; not broad public availability.
Release gate	Required evidence and approval before expanding distribution.

C Engineering and support runbook

Reproduce the current checks

From the app source folder, use a compatible Swift toolchain and Node.js. Run against disposable/local test data. The supplied script includes the Swift Testing plugin workaround required by this Mac's toolchain.

```
./scripts/test.sh
./scripts/build.sh
# Quit Habit Gate before replacing the installed bundle.
./scripts/install.sh
python3 scripts/check-install.py
python3 scripts/profile.py --seconds 65
```

Installation is a mutation, not a diagnostic. Back up the existing signed bundle and owner-only configuration first, keep backups private, and do not overwrite an active user's policy to reproduce a bug. The current working folder is not a Git repository; establish a repository and tagged release history before team development or public distribution.

Triage without exposing secrets

1. Record versions, active habit date, last successful sync, coverage state and a redacted screenshot. Confirm the distinction between selected and actually protected targets.
2. For a sync issue, check network/provider availability and the sanitized error. Respect the displayed retry/backoff; reconnect only when needed. Enter credentials only in the app's secure field and macOS prompts.
3. For Chrome, manually confirm extension loading in the intended profile, permissions, native host and private-mode setting. A successful helper CLI test alone does not prove browser loading.
4. For a selected Mac app, verify its exact bundle ID and foreground hiding. Do not terminate a user's session to claim success. Collect hide-failure status.
5. For reset/storage issues, preserve files and timestamps privately, reproduce with fixtures and require the expected/observed result. Do not edit a real journal to make a test pass.

Upgrade, rollback and account integration

Keep a known-good signed bundle, policy backup and matching extension/native-host identity. After updating or restoring, verify signature, connection, policy preservation, expiry and login registration. A local signature change may require the user to reapprove the existing Keychain item. Do not bypass that macOS prompt.

An existing Orange Pi bridge in the owner's separate project can also change Habitify data. Its current 5 AM boundary differs from Habit Gate's 4 AM boundary; between 4 and 5 AM they can refer to different dates. Habit Gate does not depend on that service, and this review does not redesign it. Inventory other consumers before generating a replacement API key because provider key rotation can revoke their access.

Maintain this document. Update baseline versions, screenshots, evidence hashes and release decisions together. Compile twice with pdfLaTeX, review every page, and label any synthetic UI state. In Overleaf, upload the source ZIP, select `main.tex` and pdfLaTeX. No shell escape or external font download is required.